



数据结构

Data Structure

许 蕾

xlei@nju.edu.cn



第五章 树与二叉树



- 树和森林的概念
- 二叉树
- 二叉树遍历
- 线索化二叉树
- 树与森林
- 堆
- Huffman树

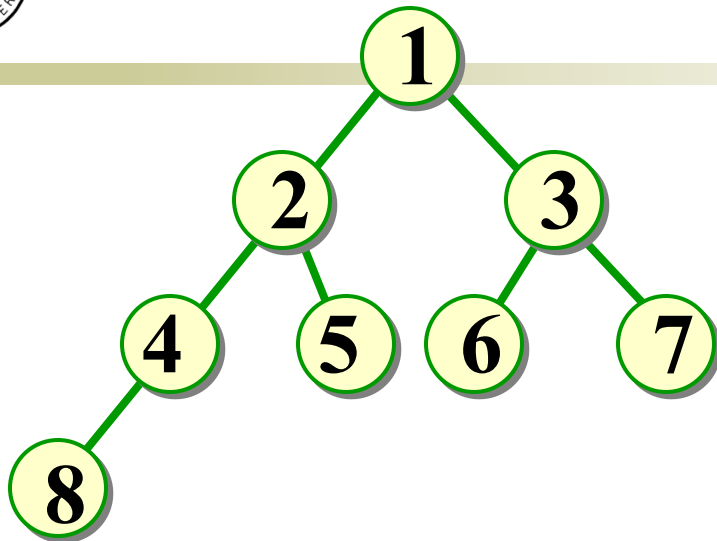


5.9 Huffman树



5.9.1 路径长度 (Path Length)

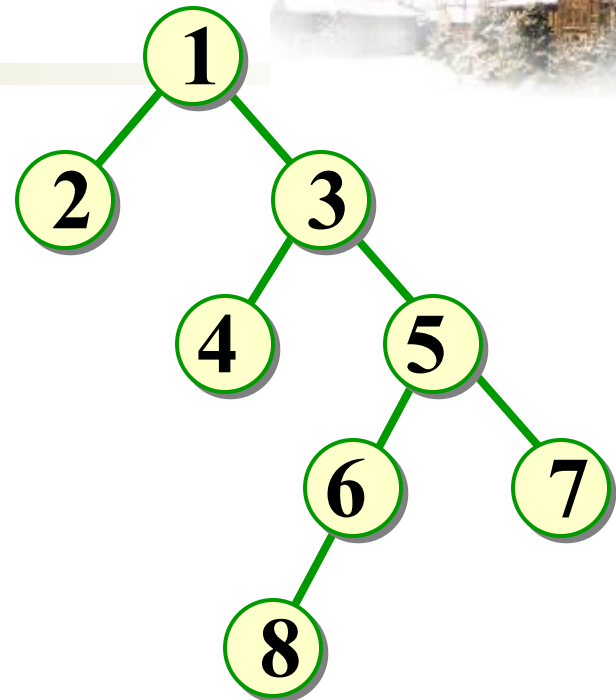
- 两个结点之间的**路径长度** PL 是连接两结点的路径上的分支数。
- 树的**外部路径长度** EPL 是各叶结点（外结点）到根结点的路径长度之和。
- 树的**内部路径长度** IPL 是各非叶结点（内结点）到根结点的路径长度之和。
- 树的路径长度 $PL = EPL + IPL$ 。



$$\text{IPL} = 0+1+1+2 = 4$$

$$\text{EPL} = 2+2+2+3 = 9$$

$$\text{PL} = 4+9=13$$



$$\text{IPL} = 0+1+2+3 = 6$$

$$\text{EPL} = 1+2+3+4 = 10$$

$$\text{PL} = 6+10=16$$



- n 个结点的二叉树的路径长度不小于下述数列前 n 项的和, 即

$$PL = \sum_{i=1}^n \lfloor \log_2 i \rfloor$$
$$= 0 + 1 + 1 + 2 + 2 + 2 + 2 + 3 + 3 + \dots$$

- 其路径长度最小者为

$$PL = \sum_{i=1}^n \lfloor \log_2 i \rfloor$$

- 完全二叉树满足这个要求。



带权路径长度

(Weighted Path Length, WPL)

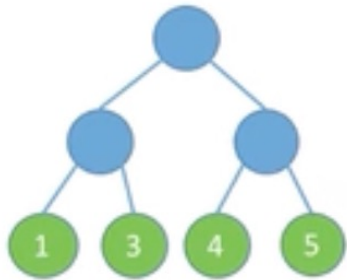
- 在很多应用问题中为树的**叶结点**赋予一个权值，用于表示出现**频度**、**概率值**等。因此，在问题处理中把叶结点定义得不同于非叶结点，把叶结点看成“外结点”，非叶结点看成“内结点”。这样的二叉树称为**相应权值的扩充二叉树**。
- 扩充二叉树中只有度为 2 的内结点和度为 0 的外结点。根据二叉树的性质，有 n 个外结点就有 $n-1$ 个内结点，总结点数为 $2n-1$ 。



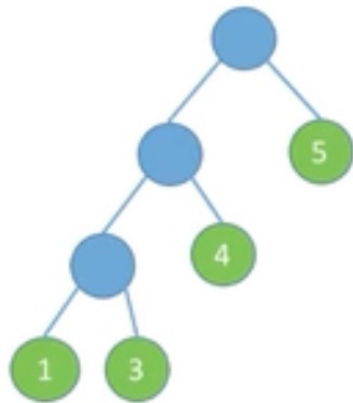
- 若一棵扩充二叉树有 n 个外结点, 第 i 个外结点的权值为 w_i , 它到根的路径长度为 l_i , 则该外结点到根的带权路径长度为 $w_i * l_i$ 。
- 扩充二叉树的带权路径长度定义为树的各**外结点到根的带权路径长度之和**。

$$WPL = \sum_{i=1}^n w_i * l_i$$

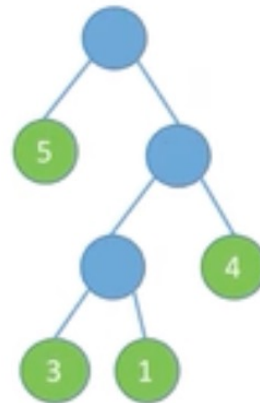
- 对于同样一组权值, 如果放在外结点上, 组织方式不同, 带权路径长度也不同。



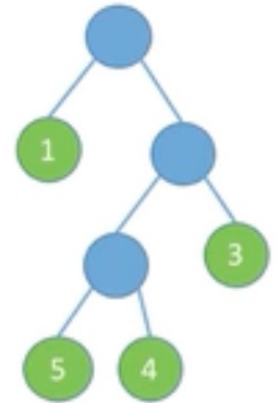
$$WPL = 2 \cdot 1 + 2 \cdot 3 + 2 \cdot 4 + 2 \cdot 5 = 26$$



$$WPL = 1 \cdot 5 + 2 \cdot 4 + 3 \cdot 1 + 3 \cdot 3 = 25$$



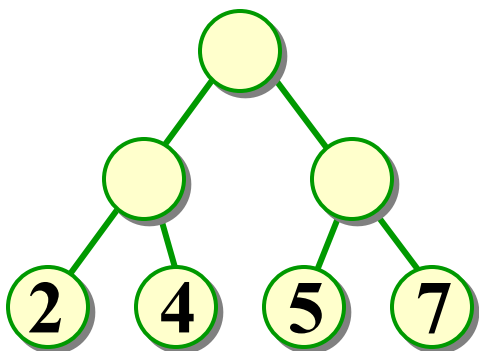
$$WPL = 1 \cdot 5 + 2 \cdot 4 + 3 \cdot 1 + 3 \cdot 3 = 25$$



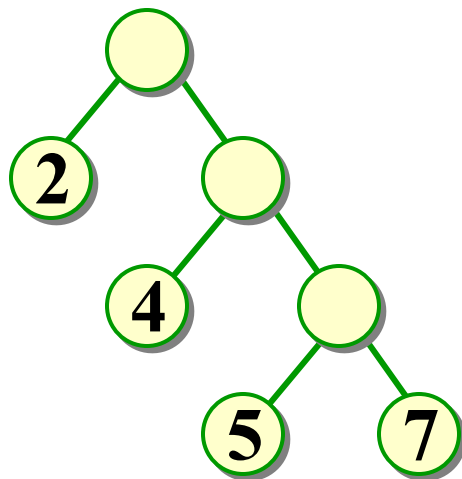
$$WPL = 1 \cdot 1 + 2 \cdot 3 + 3 \cdot 5 + 3 \cdot 4 = 34$$



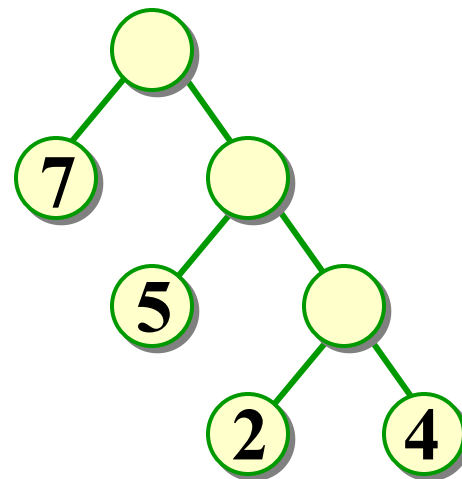
具有不同带权路径长度的扩充二叉树



$$\begin{aligned} \text{WPL} &= 2*2 + \\ &4*2 + 5*2 + \\ &7*2 = 36 \end{aligned}$$



$$\begin{aligned} \text{WPL} &= 2*1 + \\ &4*2 + 5*3 + \\ &7*3 = 46 \end{aligned}$$



$$\begin{aligned} \text{WPL} &= 7*1 + \\ &5*2 + 2*3 + \\ &4*3 = 35 \end{aligned}$$

带权路径长度达到最小



5.9.2 Huffman树



- 带权路径长度达到最小的扩充二叉树即为 Huffman树，也叫最优二叉树。压缩率通常在20%-90%。
- 在Huffman树中，权值大的结点离根最近。



Huffman树的构造算法



1. 给定 n 个权值 $\{w_0, w_1, w_2, \dots, w_{n-1}\}$, 构造具有 n 棵扩充二叉树的森林 $F = \{T_0, T_1, T_2, \dots, T_{n-1}\}$, 其中每棵扩充二叉树 T_i 只有一个带权值 w_i 的根结点, 其左、右子树均为空。
2. 重复以下步骤, 直到 F 中仅剩一棵树为止:
 - a) 在 F 中选取两棵根结点的权值最小的扩充二叉树, 做为左、右子树构造一棵新的二叉树。置新的二叉树的根结点的权值为其左、右子树上根结点的权值之和。
 - b) 在 F 中删去这两棵二叉树。
 - c) 把新的二叉树加入 F 。



Huffman树的构造过程

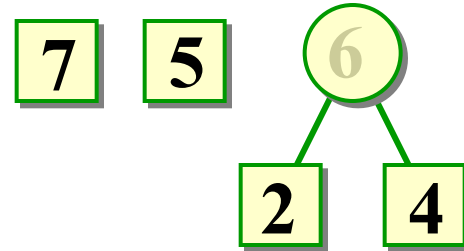


$F : \{7\} \{5\} \{2\} \{4\}$

7 5 2 4

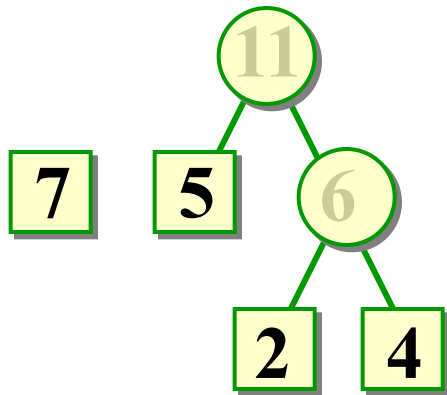
初始

$F : \{7\} \{5\} \{6\}$



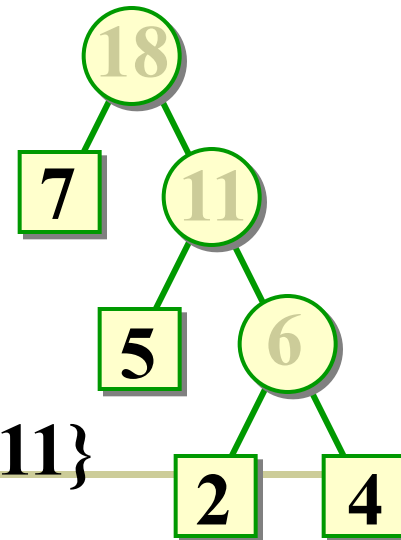
合并{2} {4}

$F : \{7\} \{11\}$



合并{5} {6}

$F : \{18\}$



合并{7} {11}



Huffman树的类定义



```
#include "heap.h"

const int DefaultSize = 20;    //缺省权值集合大小

template <class E>
struct HuffmanNode {           //树结点的类定义
    E data;                     //结点数据
    HuffmanNode<E> *parent;     //父结点指针
    HuffmanNode<E> *leftChild, *rightChild;
                                //左、右子女
    HuffmanNode () : parent(NULL), leftChild(NULL),
        rightChild(NULL) { }    //构造函数
```



```
HuffmanNode (E elem,                //构造函数
    HuffmanNode<E> *pr = NULL,
    HuffmanNode<E> *left = NULL,
    HuffmanNode<E> *right = NULL)
    : data (elem), parent (pr), leftChild (left),
      rightChild (right) { }
};
```



```
template <class E>
```

```
class HuffmanTree {
```

//Huffman树类定义

```
public:
```

```
    HuffmanTree (E w[], int n);
```

//构造函数

```
    ~HuffmanTree() {deleteTree (root);} //析构函数
```

```
protected:
```

```
    HuffmanNode<E> *root;
```

//树的根

```
    void deleteTree (HuffmanNode<E> *t);
```

//删除以 t 为根的子树

```
    void mergeTree (HuffmanNode<E>& ht1,
```

```
                    HuffmanNode<E>& ht2,
```

```
                    HuffmanNode<E> *& parent);
```

```
};
```



建立Huffman树的算法



```
template <class E>
```

```
HuffmanTree<E>::HuffmanTree (E w[], int n) {
```

```
    //给出 n 个权值w[0]~w[n-1], 构造Huffman树
```

```
    minHeap<E> hp;        //使用最小堆存放森林
```

```
    HuffmanNode<E> *parent, &first, &second;
```

```
    HuffmanNode<E> *NodeList =
```

```
        new HuffmanNode<E>[n];    //森林
```

```
    for (int i = 0; i < n; i++) {    //森林各棵树初始化
```

```
        NodeList[i].data = w[i];
```

```
        NodeList[i].leftChild = NULL;
```




```
NodeList[i].rightChild = NULL;
```

```
NodeList[i].parent = NULL;
```

```
hp.Insert(NodeList[i]); //插入最小堆中
```

```
}
```

```
for (i = 0; i < n-1; i++) { //n-1趟, 建Huffman树
```

```
    hp.RemoveMin (first); //选择根权值最小的树
```

```
    hp.RemoveMin (second); //选择根权值次小的树
```

```
    mergeTree (first, second, parent); //合并
```

```
    hp.Insert (*parent); //重新插入堆中
```

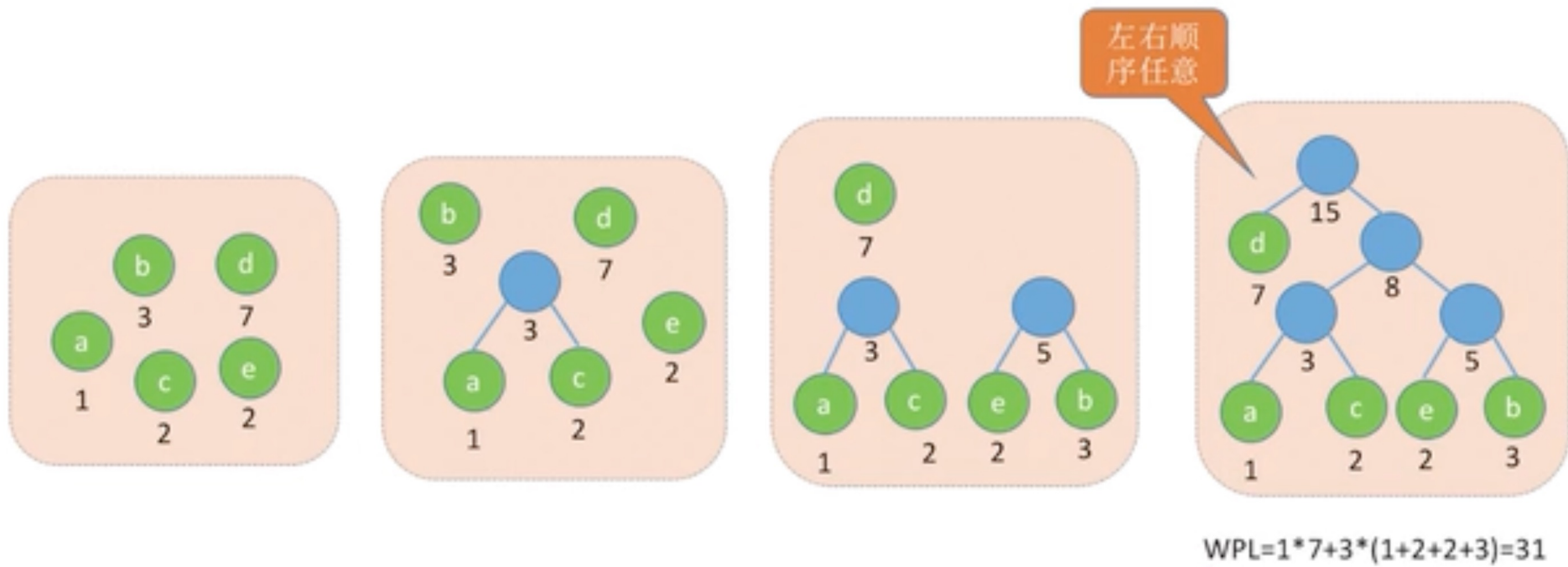
```
}
```

```
root = parent; //建立根结点
```

```
};
```



```
template <class E>
void HuffmanTree<E>::
mergeTree (HuffmanNode<E>& bt1,
           HuffmanNode<E>& bt2,
           HuffmanNode<E> *& parent) {
parent = new HuffmanNode<E>;
parent->leftChild = &bt1;
parent->rightChild = &bt2;
parent->data.key =
           bt1.root->data.key+bt2.root->data.key;
bt1.root->parent = bt2.root->parent = parent;
};
```





5.9.4 Huffman树的应用



哈夫曼编码

电报是一种最早用电的方式来传送信息的、可靠的即时远距离通信方式

报文内容: aaaaaaabbbbbccdddd

出现字符: a、b、c、d
出现次数: 7、5、2、4

等长码:
a:00 b:01
c:10 d:11

不等长码:
a:0 b:1
c:00 d:01

电文内容:
000000000000000010101
0101101011111111

按照二位一分进行译码

电文内容:
00000001111100000101
0101

无法翻译

前缀编码: 满足任一个字符的编码都不是另一个字符编码的前缀

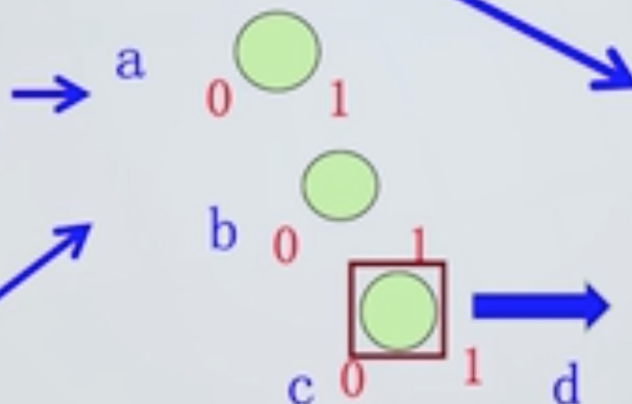


哈夫曼编码

报文内容: aaaaaaabbbbbccdddd

出现字符: a、b、c、d
出现次数: 7、5、2、4

约定:
左分支表示字符'0'
右分支表示字符'1'



不等长前缀码:
a:0
b:10
c:110 d:111

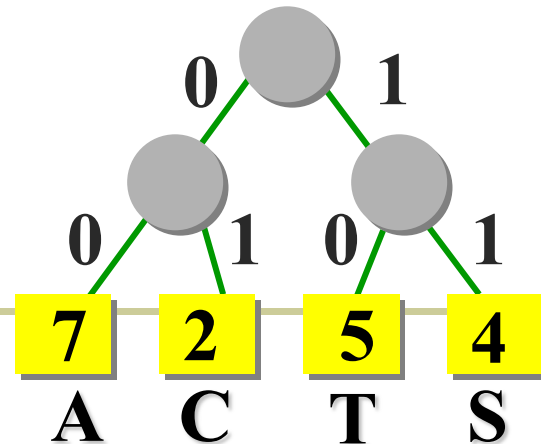
特征: 每一码不会是另一码的前缀, 译码时可惟一复原

$$WPL = \sum_{k=1}^n w_k l_k = 1 * 7 + 2 * 5 + 3 * 2 + 3 * 4 = 35 \rightarrow$$

电文内容:
000000010101010101101101
111111111111



- 主要用途是实现数据压缩。设给出一段报文：
CAST CAST SAT AT A TASA
- 字符集合是 $\{C, A, S, T\}$ ，各个字符出现的频度（次数）是 $W = \{2, 7, 4, 5\}$ 。
- 若给每个字符以等长编码（2位二进制足够）
A : 00 T : 10 C : 01 S : 11
- 则总编码长度为
 $(2+7+4+5) * 2 = 36$ 。



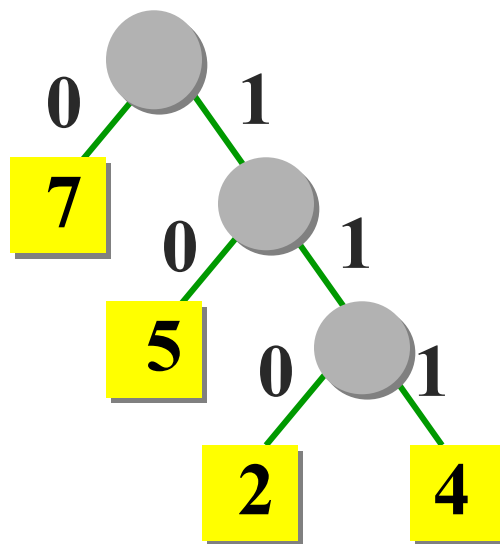


- 能否减少总编码长度，使得发出同样报文，可以用最少的二进制代码？
- 若按各个字符出现的概率不同而给予不等长编码，可望减少总编码长度。
- 各字符{ C, A, S, T }出现概率为{ $2/18$, $7/18$, $4/18$, $5/18$ }，化整为{ 2, 7, 4, 5 }。以它们为各叶结点上的权值，建立Huffman树。左分支赋0，右分支赋1，得Huffman编码(变长编码)。



A : 0 T : 10 C : 110 S : 111

- 它的总编码长度： $7*1+5*2+(2+4)*3 = 35$ 。比等长编码的情形要短。
- 总编码长度正好等于Huffman树的带权路径长度WPL。
- Huffman编码是一种前缀编码，即任一个二进制编码不是其他二进制编码的前缀。解码时不会混淆。



Huffman编码树



File :

b	p	'	m	j	o	d	a	i	r	u	l	s	e	
1	1	2	2	3	3	3	4	4	5	5	6	6	8	12

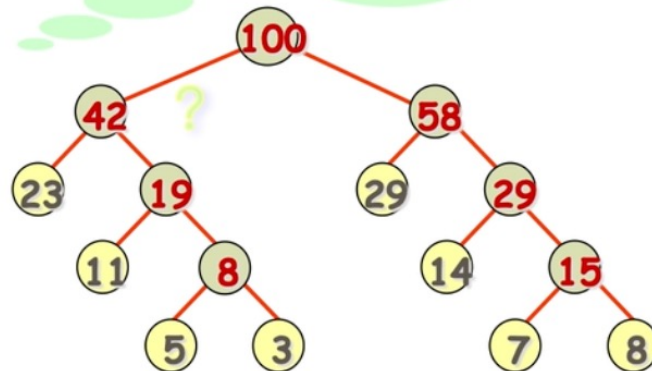




a	b	c	d	e	f	g	h
0.05	0.29	0.07	0.08	0.14	0.23	0.03	0.11



How to Minimize the length of telegraphese?



a: 0110
b: 10
c: 1110
d: 1111
e: 110
f: 00
g: 0111
h: 010

英文字母使用频率表:(%)

A 8.19	B 1.47	C 3.83	D 3.91	E 12.25	F 2.26	G 1.71
H 4.57	I 7.10	J 0.14	K 0.41	L 3.77	M 3.34	N 7.06
O 7.26	P 2.89	Q 0.09	R 6.85	S 6.36	T 9.41	
U 2.58	V 1.09	W 1.59	X 0.21	Y 1.58	Z 0.08	

试试设计哈夫曼编码，并计算数据压缩率



哈夫曼树

概念

结点的权：某种特定含义的数值

结点的带权路径长度 = 根到结点路径长度 * 结点的权值

树的带权路径长度(WPL) = 树中所有叶子结点的带权路径长度之和

哈夫曼树（最优二叉树）：在含有给定的n个带权叶结点的二叉树中，WPL 最小的二叉树

构造哈夫曼树

每次选两个根节点权值最小的树合并，并将二者权值之和作为新的根节点的权值

哈夫曼树不唯一，但WPL必然都是最小值

哈夫曼编码

将字符频次作为字符结点权值，构造哈夫曼树，即可得哈夫曼编码，可用于数据压缩

前缀编码——没有一个编码是另一个编码的前缀

固定长度编码——每个字符用相等长度的二进制位表示

可变长度编码——允许对不同字符用不等长的二进制位表示



第5章 书面作业



- (1)第247页, 5.9
 - (2)第 247 页, 5.16 :(3)(4)
 - (3)第248页, 5.20
 - 补充
- 1.森林S 对应的二叉树T的前序遍历为ABCDEFGH IJ KLMN, 中序遍历为CBDEAGHFKJMLIN。
- (1) 画出二叉树对应的森林S
 - (2) 写出T的中序线索化二叉树对应的五元组表示形式。
- 五元组表示形式如下

leftChild	ltag	data	rtag	rightChild
-----------	------	------	------	------------



2.在把一组信息M（字符串）进行哈夫曼编码后，将其哈夫曼树中最长编码的长度定义为 $\text{Maxlen}(M)$ 。若该组信息中共有 n ($n > 1$) 个不同字符需要编码（即哈夫曼树有 n 个叶子节点），求哈夫曼树的节点总数以及 $\text{Maxlen}(M)$ 的可能范围。

3.字符集 $\{a,b,c,d,e,f,g,h\}$ 的哈夫曼树如下图所示。则编码序列001011101100101000110011001的译码结果是？

